Day 89 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 13, Day 6 reference

Watch: GloVe: Global Vectors for Word Representation, Paper Overview (Scarlett's Log) | Lecture 6: GloVe, Word-word co-occurrence matrix (Kshitiz Verma)

`day89_glove_pretrained.py`, segment by segment, in the order `main()` uses things. Day 85's tagger and Day 86's corpus code are imported and explained on those days. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: X_{ij} = weighted count of word j near word i · w, w~ = word and context vectors · b, b~ = their biases · f = the weighting function

Segment 0 — Setup

Imports and settings

CODE

```
with contextlib.redirect_stdout(io.StringIO()):
    import day85_word_embeddings as d85
    import day86_word2vec_skipgram as d86
np.random.seed(42)
D = 50
WIN = 10
XMAX, ALPHA = 100.0, 0.75
LR, EPOCHS, BATCH = 0.05, 25, 4096
GLOVE_FILE = os.path.join(HERE, "day89_glove_vectors.txt.gz")
W2V_FILE = os.path.join(HERE, "day86_vectors.txt.gz")
LRS = d85.LRS
SEEDS = [0, 1, 2]
FRACTIONS = (1.0, 0.1)
```

- `import day85_word_embeddings as d85` — a script is a module: its functions and classes can be used as d85.WindowTagger.
- `contextlib.redirect_stdout(io.StringIO())` — anything printed during the import goes into a string, not the output.
- `LRS = d85.LRS` — Day 85's learning-rate grid, reused rather than copied.

Segment 1 — Counting and GloVe

cooccurrence: counting every pair once

MATH

X_{ij} = Σ *over occurrences of j within 10 words of i of 1 / distance* (*symmetric:* $X_{ij} = X_{ji}$)

CODE

```
def cooccurrence(ids, V, window=WIN):
    keys, wts = np.zeros(0, dtype=np.int64), np.zeros(0)
    for off in range(1, window + 1):
        a, b = ids[:-off].astype(np.int64), ids[off:].astype(np.int64)
        k = np.concatenate([keys, a * V + b, b * V + a])
        w = np.concatenate([wts, np.full(2 * len(a), 1.0 / off)])
        keys, inv = np.unique(k, return_inverse=True)
        wts = np.bincount(inv, weights=w)
    return keys // V, keys % V, wts
```

- `a * V + b` — one integer per (i, j) pair, so pairs can be sorted and merged like numbers.
- `np.unique(k, return_inverse=True)` — the distinct keys, and for every entry which distinct key it is.
- `np.bincount(inv, weights=w)` — sums the weights of entries with the same key: the counts, merged.
- `keys // V, keys % V` — back from one integer to (row, column).

glove_loss_grads: the weighted squared error

MATH

$J = \Sigma\, f(X_{ij})\,(w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2, \quad f(x) = \min(1, (x / 100)^{0.75})$

$dJ/dw_i = 2\, f\, e\, \tilde{w}_j, \quad dJ/d\tilde{w}_j = 2\, f\, e\, w_i, \quad dJ/db_i = dJ/d\tilde{b}_j = 2\, f\, e$

CODE

```
def glove_loss_grads(W, Wt, b, bt, i, j, x):
    f = np.minimum(1.0, (x / XMAX) ** ALPHA)
    e = (W[i] * Wt[j]).sum(axis=1) + b[i] + bt[j] - np.log(x)
    g = 2 * f * e
    return (f * e ** 2).sum(), g[:, None] * Wt[j], g[:, None] * W[i], g, g
```

- `np.minimum(1.0, (x / XMAX) ** ALPHA)` — the weight: small for rare pairs, 1 for pairs counted 100 times or more.
- `(W[i] * Wt[j]).sum(axis=1)` — a row-by-row dot product for every pair in the batch.
- `g[:, None] * Wt[j]` — g has one number per pair; [:, None] makes it a column so it scales each row.

GloVe and train_glove: AdaGrad

MATH

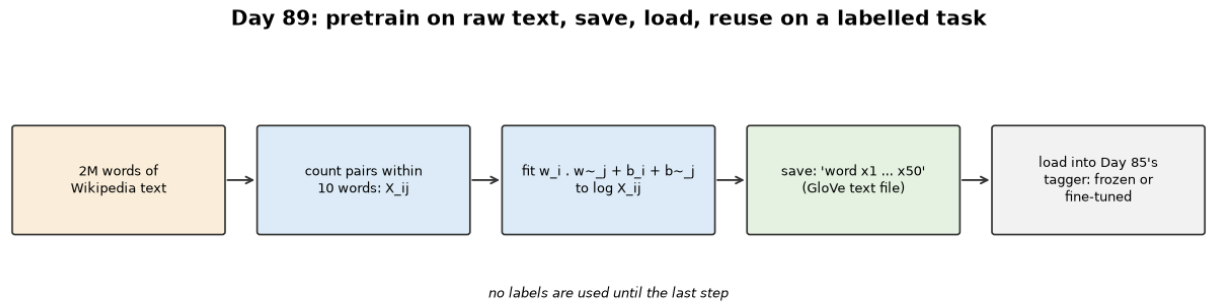
$$A \leftarrow A + g^2; \quad w \leftarrow w - lr \cdot g / \sqrt{A}$$

CODE

```
class GloVe:
    def __init__(self, V, d=D, seed=0):
        rng = np.random.RandomState(seed)
        self.W, self.Wt = (rng.rand(V, d) - 0.5) / d, (rng.rand(V, d) - 0.5) / d
        self.b, self.bt = np.zeros(V), np.zeros(V)
        self.acc = [np.ones_like(a) for a in (self.W, self.Wt, self.b, self.bt)]
    def step(self, i, j, x, lr):
        loss, *grads = glove_loss_grads(self.W, self.Wt, self.b, self.bt, i, j, x)
        for P, A, g, r in zip((self.W, self.Wt, self.b, self.bt), self.acc, grads, (i, j, i, j)):
            np.add.at(A, r, g ** 2)
            np.add.at(P, r, -lr * g / np.sqrt(A[r]))
        return loss
    def vectors(self):
        return self.W + self.Wt
def train_glove(rows, cols, X, itos, stoi, seed=0):
    model = GloVe(len(itos), seed=seed); rng = np.random.RandomState(seed + 100)
    hist = []
    for ep in range(EPOCHS):
        t0 = time.perf_counter(); perm = rng.permutation(len(X)); loss = 0.0
        for s in range(0, len(X), BATCH):
            k = perm[s:s + BATCH]
            loss += model.step(rows[k], cols[k], X[k], LR)
        hist.append((loss / len(X), time.perf_counter() - t0) + scores(model.vectors(), itos,
            stoi))
    return model, np.array(hist)
```

- `self.acc = [np.ones_like(a) for a in ...]` — AdaGrad's running sums of squared gradients, one per weight (GloVe starts them at 1).
- `zip((self.W, self.Wt, self.b, self.bt), self.acc, grads, (i, j, i, j))` — walks the four arrays, their sums, their gradients and their row indices together.
- `-lr * g / np.sqrt(A[r])` — a weight that has had large gradients takes smaller steps.

SYSTEM DIAGRAM



Pretrain on raw text, save, load, reuse on a labelled task.

Segment 2 — Vector files

save_vectors and load_vectors: the file formats

CODE

```
def save_vectors(path, words, M):
    with gzip.open(path, "wt", encoding="utf-8") as f:
        for w, v in zip(words, M):
            f.write(w + " " + " ".join(f"{x:.5f}" for x in v) + "\n")
def load_vectors(path):
    opener = gzip.open if path.endswith(".gz") else open
    with opener(path, "rt", encoding="utf-8") as f:
        lines = f.read().splitlines()
    first = lines[0].split()
    if len(first) == 2 and all(t.isdigit() for t in first):
        lines = lines[1:]
    words = [ln.split(" ", 1)[0] for ln in lines]
    M = np.array([[float(x) for x in ln.split(" ")[1:]] for ln in lines])
    return words, M
```

- `gzip.open if path.endswith(".gz") else open` — picks the function to call; both open text the same way.
- `len(first) == 2 and all(t.isdigit() for t in first)` — a word2vec header line is two integers; a GloVe line starts with a word.
- `ln.split(" ", 1)[0]` — split only at the first space: the word, then the rest of the line.

Segment 3 — Using the vectors

start_table and fit_with: pretrained vectors in Day 85's tagger

CODE

```
def start_table(ewt_itos, words, M, seed):
    E = np.random.RandomState(seed).randn(len(ewt_itos), D) * 0.1
    if M is None:
        return E, 0
    ix = {w: i for i, w in enumerate(words)}
    hit = [(r, ix[w]) for r, w in enumerate(ewt_itos) if w in ix]
    rows, src = np.array([h[0] for h in hit]), np.array([h[1] for h in hit])
    scale = np.linalg.norm(E, axis=1).mean() / np.linalg.norm(M[src], axis=1).mean()
    E[rows] = M[src] * scale
    return E, len(hit)
def fit_with(wdata, V, nt, seed, E0, train_table, lr, epochs=d85.EPOCHS):
    (win_tr, y_tr), (win_va, y_va), (win_te, y_te) = wdata
    net = d85.WindowTagger(V, nt, kind="dense" if train_table else "frozen", d=D, lr=lr,
        seed=seed)
    net.E = E0.copy()
    rng = np.random.RandomState(seed); best = (-1.0, None)
    for epoch in range(epochs):
        perm = rng.permutation(len(y_tr))
        for s in range(0, len(perm), 64):
            net.step(win_tr[perm[s:s + 64]], y_tr[perm[s:s + 64]])
        val = (net.predict(win_va) == y_va).mean()
        if val > best[0]:
            best = (val, copy.deepcopy(net))
    return best[0], (best[1].predict(win_te) == y_te).mean()
```

- `scale = ... / ...` — the pretrained rows are rescaled to the random rows' average length, so the tagger starts with inputs of the same size.
- `E[rows] = M[src] * scale` — fancy-index assignment: many table rows replaced in one statement.
- `kind="dense" if train_table else "frozen"` — Day 85's tagger already has both behaviours; frozen never updates E.

- `net.E = E0.copy()` — replace the tagger's own random table; copy() so later training never changes E0.

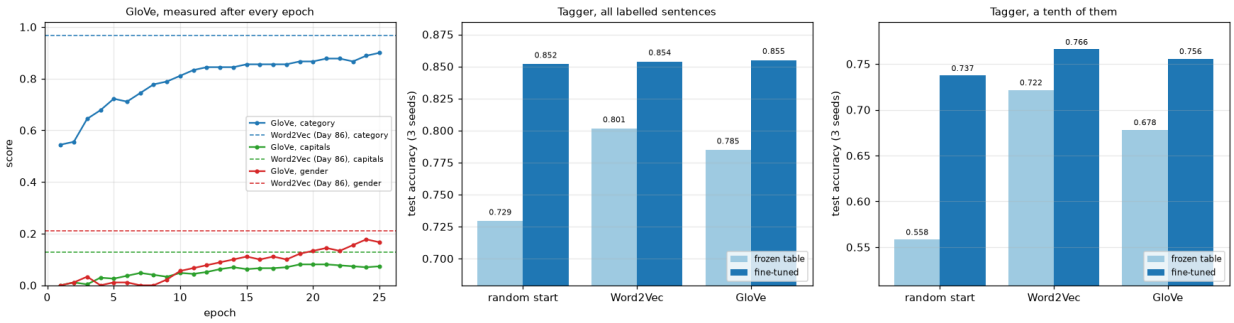
demo_tagger: twelve settings

CODE

```
for frac in FRACTIONS:
    tr = tuple(a[:int(round(len(a) * frac))] for a in data[0])
    wdata = d85.window_data([tr, data[1], data[2]])
    for src, vec in sources:
        for trained in (False, True):
            E0 = lambda s: start_table(ewt_itos, *(vec if vec else (None, None)), s)[0]
            vals = [fit_with(wdata, V, nt, 0, E0(0), trained, lr)[0] for lr in LRS]
            lr = LRS[int(np.argmax(vals))]
            tests = [fit_with(wdata, V, nt, s, E0(s), trained, lr)[1] for s in SEEDS]
            results[(frac, src, trained)] = np.mean(tests)
```

- `a[:int(round(len(a) * frac))] for a in data[0]` — the first sentences of each training array (ids, tags, mask).
- `E0 = lambda s: start_table(...)[0]` — a small function: the starting table for seed s.
- `LRS[int(np.argmax(vals))]` — the learning rate with the best validation accuracy, for this setting.

REAL GRAPH



Real graph from the script: GloVe's scores by epoch, and the tagger with each starting table.

Quick Syntax Glossary — new or reinforced constructs

- `np.unique(a, return_inverse=True)` — distinct values, and where each entry maps to.
- `np.bincount(idx, weights=w)` — sum w per index value.
- `np.searchsorted(sorted_keys, k)` — where k is (or would be) in a sorted array: a fast lookup.
- `s.split(" ", 1)` — split at the first space only.
- `import script as name` — use another lesson's functions without copying them.
- `f(x) if cond else g(x)` — a conditional expression.

End of Day 89 syntax reference. For the experiments and what they showed, see the Day 89 complete guide.